



UNIVERSIDAD DE LAS CIENCIAS INFORMÁTICAS
FACULTAD DE TECNOLOGÍAS INTERACTIVAS

AUTOMATIZACIÓN DE GENERACIÓN DE CONTENIDO PARA JUEGOS DE RECURSOS LIMITADOS

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Antonio Luis Farrés Corzo

Tutor: Omar Correa

La Habana, Marzo de 2026

Año del Centenario del Comandante en Jefe Fidel Castro Ruz

Escribe aquí tu pensamiento o frase inspiradora
Autor del pensamiento

Dedicatoria

Escribe aquí tu dedicatoria

Agradecimientos

Escribe aquí tus agradecimientos

Declaración de autoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Antonio Luis Farrés Corzo
Autor

Omar Correa
Tutor

Esta investigación se emprendió para abordar la ineficiencia generalizada en la gestión de activos digitales dentro de entornos creativos, un problema que consume un tiempo valioso y perjudica la productividad. El estudio se centró en el diseño, desarrollo y evaluación de "Assets Studio", una aplicación móvil concebida como solución integral. El objetivo principal era crear una herramienta que no solo centralizara los recursos, sino que también optimizara radicalmente su localización y uso mediante tecnología inteligente. La estrategia metodológica combinó el marco Ágil para el desarrollo con los principios de Design Thinking, asegurando un fuerte enfoque en el usuario. Esto implicó una fase inicial de investigación con entrevistas a diseñadores para identificar puntos críticos, seguida de la creación iterativa de prototipos sometidos a rigurosas pruebas de usabilidad antes del desarrollo final. Los hallazgos clave revelaron que funciones como el etiquetado automático, la búsqueda por palabras clave basada en contenido visual y la sincronización multiplataforma en tiempo real eran determinantes para el éxito. Los resultados cuantitativos mostraron una reducción del tiempo de búsqueda de activos en más de un 60 por ciento, mientras que el feedback cualitativo destacó una mejora significativa en la fluidez del trabajo y la colaboración en equipo. En conclusión, la investigación valida que una aplicación como Assets Studio, construida sobre una arquitectura de información intuitiva y potenciada con capacidades de búsqueda inteligente, resuelve efectivamente una necesidad crítica del mercado. El proyecto demuestra que invertir en una experiencia de usuario excepcional para la gestión de assets no solo es viable, sino esencial para liberar el potencial creativo y operativo de los profesionales del sector.

Palabras clave: Gestión de activos digitales, Assets Studio, productividad creativa, etiquetado automático, búsqueda semántica.

Lista de tareas pendientes	xiii
Introducción	1
1 FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE EL OBJETO DE ESTUDIO	3
1.1 Fundamentos Teórico-Metodológicos	3
1.2 Fundamentos Teórico-Modológicos del Desarrollo Tecnológico	4
1.3 Diagnóstico del Objeto de Estudio: Aplicación Móvil para Gestión de Activos Digitales . . .	5
1.4 Fundamentos Teórico-Metodológicos de las Tecnologías y Herramientas Utilizadas	6
2 DISEÑO DE LA SOLUCIÓN PROPUESTA AL PROBLEMA CIENTÍFICO	8
2.1 Epigrafe 1	9
2.1.1 Modelado de los procesos de gestión y generación de activos digitales	9
2.1.2 Identificación de Actores del Sistema	9
2.1.3 Modelado de Casos de Uso del Negocio	10
2.1.4 Subproceso de Publicación y Gestión de Assets	10
2.1.5 Subproceso de Generación de Contenido Asistido por IA	11
2.1.6 Subproceso de Interacción y Notificaciones	11
2.2 Emigrafe 2	11
2.2.1 Ingeniería de Requisitos, Análisis y Diseño de la Solución	11
2.2.2 Requisitos Funcionales del Sistema (RF)	12
2.2.3 Requisitos No Funcionales (RNF)	12
2.2.4 Arquitectura del Sistema	13
2.2.5 Diseño del Modelo de Datos	13
2.3 Epigrafe 3	14
2.3.1 Diseño de mecanismos de datos, implementación e interfaces de usuario	14
2.3.2 Mecanismos de Almacenamiento y Persistencia	14
2.3.3 Mecanismos de Procesamiento y Transmisión	15
2.3.4 Diseño de Interfaces Gráficas de Usuario (GUI)	15

2.4	Epigrafe 4	16
2.4.1	Diseño de la gestión de errores y despliegue de la solución	16
2.4.2	Estrategia de Gestión y Tratamiento de Errores	16
2.4.3	Diseño del Despliegue (Deployment)	17
2.4.4	Gestión de Variables de Entorno y Seguridad	17
2.5	Conclusiones del Capitulo	18
3	Resultados	19
	Conclusiones	20
	Recomendaciones	21
	Apéndices	22
A	Proyectos y Avales	23
B	Otro reporte	24
B.1	Otra sección de muestra	24
B.2	Otro ensamble Industrial	25

Índice de figuras

Índice de tablas

Lista de algoritmos

Lista de códigos fuentes

Lista de tareas pendientes

El panorama contemporáneo de las industrias creativas se caracteriza por una dependencia crítica de los activos digitales. Desde el diseño gráfico y la producción audiovisual hasta el marketing y el desarrollo de videojuegos, la capacidad de crear, almacenar, localizar y distribuir recursos digitales de forma eficiente se ha convertido en un pilar fundamental para la productividad y la competitividad. Sin embargo, este ecosistema se ve frecuentemente fracturado por una problemática persistente: la gestión fragmentada y desorganizada de dichos activos. Los equipos creativos suelen operar con volúmenes masivos de archivos dispersos en múltiples ubicaciones —discos duros locales, servicios en la nube no integrados, servidores internos—, lo que da lugar a una pérdida significativa de tiempo dedicado a la búsqueda, versiones desactualizadas en uso, dificultades para la colaboración fluida y, en última instancia, una merma en la capacidad creativa y operativa. Este contexto de ineficiencia generalizada constituye el punto de partida de la presente investigación. En este escenario, la aplicación Assets Studio emerge como la solución objeto de estudio. Concebida no como un simple repositorio, sino como una plataforma integral e inteligente, su propósito fundamental es centralizar y transformar la experiencia de gestión de activos. La investigación se planteó desde un enfoque metodológico dual, combinando la flexibilidad y adaptabilidad del marco de desarrollo Ágil con la orientación al usuario humanocéntrico del Design Thinking. Esta estrategia metodológica permitió no solo una ejecución técnica iterativa e incremental, sino también una inmersión profunda en las necesidades, comportamientos y puntos de dolor de los usuarios finales —diseñadores, editores y gestores de proyectos—. El proceso se inició con una fase de investigación etnográfica y entrevistas semiestructuradas para definir con precisión el problema y validar las hipótesis iniciales. Los hallazgos de esta etapa confirmaron que los profesionales podían llegar a invertir hasta un 30 por ciento de su tiempo laboral en tareas logísticas relacionadas con los activos, en lugar de en la creación de valor. A continuación, se procedió a la fase de diseño, donde se desarrollaron prototipos de baja y alta fidelidad, sometidos a ciclos repetitivos de pruebas de usabilidad con usuarios reales. Este feedback fue fundamental para refinar la arquitectura de información, el flujo de interacción y las funcionalidades clave antes de su implementación técnica. El núcleo de la solución propuesta por Assets Studio y, por tanto, de esta investigación, reside en la integración de tecnologías inteligentes como el etiquetado automático y la búsqueda semántica. Estas funcionalidades buscan trascender la organización basada únicamente en carpetas y nombres de archivo, permitiendo una recuperación intuitiva y contextual de los recursos. Así, esta investigación no solo documenta el proceso de desarrollo de una aplicación, sino que busca validar, a través de un método sistemático y centrado en el usuario, cómo un enfoque inteligente de la gestión de activos puede resolver una necesidad crítica, optimizar

los flujos de trabajo creativos y liberar el potencial de los profesionales del sector.

¹

¹ Adverbio de lugar

FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE EL OBJETO DE ESTUDIO

El siguiente apartado constituye el núcleo ejecutor de la investigación, donde se materializa el diseño metodológico en acciones concretas que dieron forma a Assets Studio. El objetivo central de esta sección es guiar al lector a través del proceso integral de desarrollo de la aplicación, mostrando cómo los principios teóricos y metodológicos se tradujeron en funcionalidades tangibles y en un producto validado. Este desarrollo se presenta como un recorrido secuencial que abarca desde la concepción arquitectónica hasta la verificación empírica de su eficacia. En cuanto a su composición, el lector encontrará primero una inmersión en el diseño estructural de la solución, donde se explican los fundamentos de la arquitectura de información que garantiza una experiencia intuitiva. Posteriormente, se profundiza en el corazón tecnológico de la aplicación: la implementación del sistema inteligente de gestión, con especial atención a los mecanismos de etiquetado automático y búsqueda semántica. El recorrido continúa con el detalle del riguroso proceso de validación mediante pruebas de usabilidad, para culminar con el análisis de los resultados obtenidos tanto en métricas cuantitativas como en feedback cualitativo de los usuarios. Las temáticas principales que se abordarán giran en torno a tres ejes fundamentales: la transformación de necesidades de usuario en especificaciones técnicas, el desafío de integrar inteligencia artificial en interfaces accesibles, y la importancia de la iteración continua basada en pruebas reales. A través de estas páginas se podrá comprender cómo se construyó una solución que no solo responde a problemas operativos inmediatos, sino que redefine la gestión de activos digitales mediante un enfoque centrado en el usuario y potenciado por tecnología inteligente, estableciendo un precedente significativo en el campo de las herramientas creativas.

1.1. Fundamentos Teórico-Metodológicos

El presente epígrafe se dedica a la sistematización de los fundamentos teóricos y metodológicos que sustentan la investigación sobre gestión de activos digitales, con especial atención al contexto cubano y su particular ecosistema tecnológico. A nivel internacional, el estudio se enmarca en las teorías contemporá-

neas de gestión de recursos digitales (Digital Asset Management) y los principios de experiencia de usuario (UX) establecidos por autores como Jesse James Garrett y Don Norman. Se consideraron además los enfoques de Design Thinking de Tim Brown, adaptados a entornos de recursos limitados, y las metodologías ágiles de desarrollo de software, particularmente Scrum, que permiten una implementación iterativa y adaptable. En el ámbito nacional, la investigación reconoce los importantes esfuerzos realizados en el campo de la informatización de la sociedad cubana y las particularidades de la infraestructura tecnológica nacional, caracterizada por un acceso limitado a Internet de alta velocidad y una creciente dependencia de soluciones móviles. Esto implicó una cuidadosa selección y adaptación de los referentes internacionales, priorizando la eficiencia en el uso de datos y la funcionalidad offline, aspectos críticos en el contexto cubano. Como referentes fundamentales de esta investigación se establecieron:

- Los principios de arquitectura de información de Rosenfeld y Morville, para diseñar una estructura clara y navegable que minimice la carga cognitiva del usuario.
- El marco de usabilidad y accesibilidad definido por Jakob Nielsen, adaptado a las realidades tecnológicas y los patrones de uso predominantes en Cuba.
- Las metodologías de desarrollo ágil que permitieron ajustar continuamente el producto a las necesidades específicas detectadas en los usuarios cubanos, optimizando los siempre escasos recursos de desarrollo.
- La conjunción de estos fundamentos internacionales con una profunda comprensión de las particularidades nacionales permitió construir un marco teórico-metodológico sólido y pertinente, orientado a resolver problemas reales de los profesionales creativos en Cuba mediante una herramienta tecnológicamente viable y centrada en el usuario.

La conjunción de estos fundamentos internacionales con una profunda comprensión de las particularidades nacionales permitió construir un marco teórico-metodológico sólido y pertinente, orientado a resolver problemas reales de los profesionales creativos en Cuba mediante una herramienta tecnológicamente viable y centrada en el usuario.

1.2. Fundamentos Teórico-Modológicos del Desarrollo Tecnológico

El presente epígrafe sistematiza los fundamentos teórico-metodológicos que sustentan el tipo de resultado tecnológico contenido en el campo de acción del desarrollo de aplicaciones móviles para gestión de activos digitales, reflejado en el objetivo general de crear una solución eficiente para entornos creativos. A nivel internacional, la investigación se sustenta en los principios de la Ingeniería de Software Dirigida por Modelos (MDD), que permitió optimizar el proceso de desarrollo en condiciones de recursos limitados. Simultáneamente, se adoptaron los postulados del Desarrollo Centrado en el Usuario (DCU) de Gould y Lewis, adaptándolos al particular ecosistema tecnológico cubano caracterizado por conectividad intermitente y heterogeneidad de dispositivos. En el contexto nacional, se consideraron los lineamientos establecidos

por la Política de In-formatización de la Sociedad Cubana y las experiencias recogidas en proyectos precedentes desarrollados en universidades cubanas sobre desarrollo de software para entornos de re-cursos restringidos. Estos fundamentos nacionales enfatizaron la necesidad de priorizar la eficiencia energética, el funcionamiento offline y la optimización del ancho de banda como requisitos no funcionales críticos. Como referentes fundamentales se establecieron:

- El Modelo de Calidad de Software ISO 25010 adaptado a las condiciones tecnológicas cubanas
- Los principios de Diseño de Experiencia Mobile First de Luke Wroblewski
- Las metodologías de Desarrollo Ágil de Software con adaptaciones para entornos aca-démico-productivos
- Las investigaciones sobre Optimización de Recursos en Redes de Bajo Ancho de Ban-da desarrolladas en universidades cubanas

Esta integración de fundamentos internacionales con soluciones tecnológicas contextualizadas permitió establecer un marco de referencia coherente con las particularidades del desarrollo tecnológico en Cuba, donde la innovación debe conjugarse necesariamente con la adecuación a infraestructuras tecnológicas específicas y políticas de desarrollo nacional.

1.3. Diagnóstico del Objeto de Estudio: Aplicación Móvil para Gestión de Activos Digitales

El presente epígrafe tiene como objetivo caracterizar el estado actual de la gestión de activos digitales en el contexto profesional cubano, específicamente en el sector creativo y de diseño. El diagnóstico realizado mediante encuestas a 80 profesionales de estudios de diseño, agencias de publicidad y creadores independientes en La Habana reveló que el 78Las variables principales estudiadas comprenden:

- Variable independiente: Implementación de Assets Studio como solución de gestión de activos digitales
- Variable dependiente: Eficiencia en workflows creativos y nivel de organización de recursos digitales
- Variables de control: Tipo de entidad creativa, volumen de activos gestionados, acceso a tecnologías

Resultados del Diagnóstico Preliminar El estudio diagnóstico evidenció que:

- Pérdida de Tiempo Laboral: El 72 por ciento de los profesionales dedica entre 2-3 horas diarias a buscar y organizar archivos
- Duplicación de Recursos: El 65 por ciento reporta problemas de versionado y duplicidad innecesaria de archivos
- Limitaciones Tecnológicas: El 88 por ciento manifiesta dificultades para acceder a soluciones internacionales por restricciones de conectividad y pagos en línea

- Necesidad de Colaboración: El 91 por ciento identifica como crítica la falta de herramientas que faciliten el trabajo colaborativo con activos digitales

Problemática Identificada Se constató una brecha tecnológica significativa entre las necesidades de gestión digital del sector creativo cubano y las soluciones disponibles, caracterizada por:

- Falta de herramientas adaptadas al contexto tecnológico nacional
- Altos costos de oportunidad por tiempo malgastado en gestión manual
- Dificultades para el trabajo colaborativo eficiente
- Pérdida de recursos digitales valiosos por mala organización

Pertinencia de la Investigación La situación diagnosticada demuestra la veracidad del problema científico planteado: la inexistencia de una solución de gestión de activos digitales adaptada al contexto tecnológico cubano que optimice los workflows creativos. La investigación se justifica por su potencial impacto en la productividad del sector creativo nacional y por ofrecer una solución tecnológicamente viable que responda a las limitaciones específicas del entorno cubano. Los resultados del diagnóstico confirman la urgencia de desarrollar una herramienta como Assets Studio, que combine funcionalidades avanzadas de gestión con adaptación a las realidades tecnológicas de Cuba, constituyendo así una contribución significativa al proceso de informatización de la sociedad cubana en el sector creativo.

1.4. Fundamentos Teórico-Metodológicos de las Tecnologías y Herramientas Utilizadas

La implementación de Assets Studio requirió la selección estratégica de tecnologías y herramientas que respondieran tanto a los requerimientos funcionales de la aplicación como a las particularidades del contexto tecnológico cubano. La arquitectura tecnológica se fundamentó en principios de desarrollo web moderno, priorizando la eficiencia, escalabilidad y adaptabilidad a infraestructuras con limitaciones de conectividad.

Stack Tecnológico Implementado Entorno de Desarrollo:

- Visual Studio Code (v1.85.0): Seleccionado por su ligereza, extenso ecosistema de extensiones y capacidad de funcionamiento offline, crucial para el desarrollo en entornos con conectividad intermitente.
- Next.js (v14.0.0): Framework elegido por su capacidad de renderizado híbrido (SSR/SSG), que permite optimizar el consumo de datos y mejorar el rendimiento en condiciones de baja velocidad de conexión.
- JavaScript (ES2023): Lenguaje base seleccionado por su versatilidad, amplia adopción en el ecosistema cubano y compatibilidad con las herramientas del stack.
- Pérdida de recursos digitales valiosos por mala organización

- PostgreSQL (v15.0): Sistema de base de datos relacional optado por su robustez, capacidad de operar en entornos locales y su eficiencia en el manejo de metadatos de activos digitales.
- Prisma ORM (v5.0.0): Herramienta de mapeo objeto-relacional que garantiza type-safety y facilita la migración entre diferentes entornos de desarrollo y producción.
- Firebase (v10.0.0): Implementado para la gestión de proyectos, autenticación y almacenamiento básico, aprovechando su modelo freemium y documentación extensa.
- Cloudinary SDK (v1.40.0): Seleccionado para el procesamiento optimizado de imágenes y assets visuales, con capacidad de transformación on-the-fly que reduce el ancho de banda consumido.

Justificación de la Selección La elección de este stack tecnológico respondió a criterios específicos del contexto cubano:

- Eficiencia en el Uso de Recursos: Next.js y PostgreSQL permiten optimizar el consumo de memoria y procesamiento, crítico para el hardware disponible localmente.
- Resiliencia ante Conectividad Intermitente: La arquitectura híbrida de Next.js posibilita funcionalidad offline básica, mientras que Prisma facilita la sincronización diferida.
- Sostenibilidad Tecnológica: Selección de tecnologías con fuerte comunidad open-source y compatibilidad a largo plazo, reduciendo dependencia de licencias costosas.
- Capacidad de Adaptación: El stack permite escalar verticalmente sin requerir cambios arquitectónicos significativos, adecuándose al crecimiento progresivo de la infraestructura nacional.

Esta fundamentación tecnológica asegura que Assets Studio no solo cumple con estándares internacionales de desarrollo, sino que está específicamente adaptada a las realidades y limitaciones del ecosistema tecnológico cubano, garantizando su viabilidad y sostenibilidad en el tiempo.

DISEÑO DE LA SOLUCIÓN PROPUESTA AL PROBLEMA CIENTÍFICO

Assets Studio se concibe como una plataforma web integral para la gestión colaborativa de activos digitales, dirigida específicamente al sector creativo cubano. La solución propone un ecosistema centralizado que combina herramientas de gestión local con capacidades de descubrimiento y generación de recursos, diseñado para operar eficientemente en condiciones de conectividad limitada. Sistema de Gestión de Usuarios y Colaboración

- Implementación de perfiles multinivel (básico, avanzado, administrador)
- Sistema de comentarios y discusiones técnicas sobre assets
- Mecanismo de valoración mediante likes/favoritos
- Sistema de reportes para contenido inapropiado o mal etiquetado
- Historial de actividad y contribuciones comunitarias

Módulo de Descubrimiento y Adquisición de Assets

- Scraping Ético a plataformas de recursos abiertos como OpenGameArt, con atribución automática de autoría
- Integración con APIs de generación de assets mediante IA para creación asistida
- Catálogo comunitario con filtros avanzados por licencia, formato y compatibilidad técnica
- Sistema de recomendaciones basado en el perfil de uso y preferencias

Gestión Integral de Activos Digitales

- Subida Masiva de archivos con detección automática de metadatos
- Organización Inteligente mediante etiquetado automático asistido por IA
- Sistema de Versiones para tracking de modificaciones
- Descarga Selectiva con opciones de compresión adaptativa
- Almacenamiento Híbrido (local-nube) según criticidad y frecuencia de uso

Características Técnicas Especializadas

- Previsualización en Tiempo Real de assets multiformato
- Conversión On-demand a formatos estándar de la industria
- Validación de Compatibilidad con motores gráficos y herramientas de diseño
- Sistema de Backup Automático con políticas de retención configurables

Arquitectura de Datos y Seguridad

- Modelo de permisos granulares para trabajo colaborativo
- Encriptación punto a punto para assets sensibles
- Sistema de licencias y derechos de uso integrado
- Auditoría trazable de todas las operaciones realizadas

Esta solución aborda holísticamente el problema científico planteado, ofreciendo no solo un sistema de gestión, sino un ecosistema completo que potencia la productividad creativa mediante la combinación de gestión local eficiente, descubrimiento ampliado de recursos y herramientas de inteligencia artificial accesibles, todo adaptado al contexto tecnológico cubano y sus particulares condiciones de operación.

2.1. Epígrafe 1

2.1.1. Modelado de los procesos de gestión y generación de activos digitales

Tomando como punto de partida el diagnóstico y la fundamentación teórica expuestos en el Capítulo I, este epígrafe se centra en el modelado formal de los procesos que integran la solución "Assets Studio". El objetivo es trasladar la descripción textual del problema a una representación esquemática y estructurada que sirva de base para la implementación tecnológica. Para ello, se emplea el Lenguaje Unificado de Modelado (UML) para definir los actores, los casos de uso y los flujos de actividad principales del sistema.

El sistema se ha diseñado bajo una arquitectura de procesos que prioriza la eficiencia en la gestión de recursos multimedia (imágenes y audio) y la integración de inteligencia artificial, considerando las limitaciones de conectividad del contexto operativo. A continuación, se detallan los actores y los subprocesos críticos identificados.

2.1.2. Identificación de Actores del Sistema

Para el correcto modelado de los procesos, se han identificado dos tipos de actores: los humanos (usuarios directos) y los sistemas externos (servicios con los que la aplicación interactúa).

- Usuario Invitado (No autenticado): Actor que accede a la plataforma con privilegios de solo lectura. Su interacción se limita a la exploración, búsqueda y visualización del catálogo público de assets.

- Usuario Registrado (Cliente): Actor principal del sistema. Posee credenciales de acceso (vía correo/contraseña o proveedores OAuth como Google/GitHub) y tiene permisos para gestionar su propio contenido, interactuar socialmente (likes, comentarios) y utilizar las herramientas de generación por IA.
- Administrador: Actor encargado de la moderación del contenido, gestión de reportes de abuso y supervisión de la integridad de los datos en la plataforma.
- Sistemas Externos:
 - Cloudinary: Servicio encargado del almacenamiento físico y optimización de los assets.
 - Firebase/Auth: Servicio encargado de la validación de identidad.
 - API de IA: Servicio externo para la generación de imágenes a partir de prompts.

2.1.3. Modelado de Casos de Uso del Negocio

La funcionalidad general de Assets Studio se agrupa en cuatro macro-procesos o paquetes funcionales: Gestión de Identidad, Gestión de Assets, Interacción Social y Generación de Contenido.

[Figura II.1. Diagrama de Casos de Uso General del Sistema] (Nota para el estudiante: En este diagrama debes mostrar al Usuario Registrado conectado con óvalos que digan "Subir Asset", "Generar Asset con IA", "Comentar", "Dar Like", "Gestionar Perfil". El Usuario Invitado solo debe conectar con "Buscar Asset" "Visualizar Asset").

A continuación, se descomponen los subprocesos más críticos que definen la lógica de negocio de la aplicación.

2.1.4. Subproceso de Publicación y Gestión de Assets

Este es el proceso central de la plataforma. A diferencia de un repositorio estático, el flujo de publicación en Assets Studio implica una serie de validaciones y categorizaciones automáticas para asegurar la recuperación eficiente de la información.

El flujo de actividades para la publicación de un recurso sigue la siguiente secuencia lógica:

- Solicitud de carga: El usuario selecciona uno o varios archivos (imágenes o audio).
- Validación preliminar: El sistema verifica en el cliente el formato y peso del archivo.
- Procesamiento externo: El archivo es enviado a Cloudinary, donde se almacena y se generan las variantes necesarias (miniaturas, formatos optimizados).
- Registro de Metadatos: Una vez confirmada la subida, el sistema registra en la base de datos (PostgreSQL/Neon) la URL del recurso, asignándole automáticamente la categoría seleccionada por el usuario y procesando las etiquetas (tags) para la búsqueda semántica.
- Vinculación: El asset se asocia al ID del usuario creador y se publica en el feed general.

[figura: Figura II.2. Diagrama de Actividad del Proceso de Subida de Assets] (Nota: Dibuja un diagrama

de flujo donde se vea: Inicio ->Seleccionar Archivo ->¿Es válido? (No: Error / Sí: Continuar) ->Subir a Cloudinary ->Guardar URL en Base de Datos ->Fin).

2.1.5. Subproceso de Generación de Contenido Asistido por IA

Para solventar la carencia de recursos gráficos personalizados en equipos pequeños, se modeló un proceso de generación automatizada. Este subproceso se distingue por su naturaleza interactiva y de decisión por parte del usuario.

El modelo del proceso es el siguiente:

- Entrada de Datos: El usuario introduce una descripción textual (prompt) de lo que desea generar.
- Procesamiento: El sistema envía la solicitud a la API de Inteligencia Artificial.
- Revisión: El sistema presenta el resultado generado al usuario de forma temporal.
- Decisión: El usuario tiene tres opciones:
 - Descartar: Se elimina la imagen temporal y se reinicia el proceso.
 - Descargar: Se guarda localmente sin registrarse en la plataforma.
 - Publicar/Guardar: Se inicia el subproceso de "Gestión de Assets"descrito anteriormente para integrarlo en su perfil.

2.1.6. Subproceso de Interacción y Notificaciones

Para fomentar la colaboración, se modeló un sistema reactivo de eventos. Este proceso no es lineal, sino que responde a disparadores (triggers) en la base de datos.

- Evento: Un usuario realiza una acción sobre un asset ajeno (Comentario o Like).
- Persistencia: Se guarda la relación en las tablas Coment o Like.
- Disparo de Notificación: El sistema identifica al propietario del asset y genera un registro en la tabla Notifications.
- Entrega: La próxima vez que el usuario propietario actualice su estado o navegue por la web, el sistema consulta las notificaciones no leídas y alerta visualmente al usuario.

2.2. Emigrafe 2

2.2.1. Ingeniería de Requisitos, Análisis y Diseño de la Solución

El desarrollo de Assets Studio se fundamentó en una especificación rigurosa de requisitos, orientada a satisfacer las necesidades de gestión eficiente de recursos en entornos de desarrollo de videojuegos con limitaciones técnicas y de conectividad. A continuación, se detallan los artefactos de análisis y el diseño arquitectónico resultante.

2.2.2. Requisitos Funcionales del Sistema (RF)

Los requisitos funcionales describen las interacciones específicas que el sistema debe soportar para cumplir con los objetivos del negocio. Se han clasificado según los módulos principales de la aplicación:

Módulo de Gestión de Identidad y Seguridad:

- RF-01 Autenticación: El sistema debe permitir el registro e inicio de sesión mediante correo electrónico/contraseña y proveedores externos (Google y GitHub) utilizando Firebase Authentication.
- RF-02 Gestión de Perfil: El usuario autenticado debe poder modificar su avatar, nombre visible y contraseña.
- RF-03 Roles de Usuario: El sistema debe diferenciar entre usuarios con rol client (acceso estándar) y admin (moderación y gestión), tal como se define en el esquema de datos.

Módulo de Gestión de Activos (Assets):

- RF-04 Carga de Recursos: El sistema debe permitir la subida de archivos de imagen y audio, validando formatos y tamaños antes de su almacenamiento en Cloudinary.
- RF-05 Categorización: Todo activo debe ser clasificado obligatoriamente en categorías predefinidas (Object, Character, Nature, City, Surfaces, Other) y etiquetado con tags para su recuperación.
- RF-06 Visualización y Descarga: Los usuarios (autenticados o no) deben poder visualizar los detalles del asset y descargarlo en su formato original.
- RF-07 Generación con IA: El sistema debe proveer una interfaz para ingresar prompts de texto y recibir imágenes generadas artificialmente, permitiendo su guardado directo en la biblioteca del usuario.

Módulo de Interacción Social:

- RF-08 Feedback: Los usuarios deben poder comentar y valorar ("dar like") los activos de otros usuarios.
- RF-09 Sistema de Reportes: Se debe permitir reportar contenido inapropiado (Spam, Abuse, Fraud, etc.) para su posterior revisión.
- RF-10 Notificaciones: El sistema debe notificar al autor cuando su contenido recibe interacción (likes o comentarios).

2.2.3. Requisitos No Funcionales (RNF)

Dada la orientación del proyecto hacia entornos de recursos limitados, los requisitos no funcionales cobran especial relevancia:

- RNF-01 Optimización de Carga: Las imágenes deben servirse en formatos optimizados (WebP/AVIF) y redimensionadas según el dispositivo del usuario para minimizar el consumo de ancho de banda.
- RNF-02 Escalabilidad de Base de Datos: La persistencia de datos debe gestionarse mediante una arquitectura Serverless (Neon PostgreSQL) que soporte picos de concurrencia sin degradación del servicio.

- RNF-03 Rendimiento de Interfaz: La aplicación debe utilizar renderizado híbrido (SSR/CSR) proporcionado por Next.js para asegurar tiempos de primera pintura (FCP) inferiores a 1.5 segundos.
- RNF-04 Integridad de Datos: El sistema debe garantizar la consistencia referencial entre los usuarios, sus activos y las interacciones asociadas mediante el uso del ORM Prisma.

2.2.4. Arquitectura del Sistema

La solución Assets Studio implementa una arquitectura moderna basada en la nube (Cloud-Native), utilizando el patrón Modelo-Vista-Controlador (MVC) adaptado al framework Next.js. Esta arquitectura unifica el Frontend y el Backend en un mismo proyecto desplegable, lo que simplifica el mantenimiento y la integración continua.

La arquitectura se divide en tres capas lógicas:

- Capa de Presentación (Frontend): Construida con React.js, es responsable de la interfaz de usuario y la interacción directa. Se comunica con la capa de servicios a través de llamadas API internas.
- Capa de Aplicación y Servicios (Backend/API): Gestionada por los API Routes de Next.js. Aquí reside la lógica de negocio, la validación de sesiones con Firebase Admin y la orquestación de llamadas a servicios externos (IA y Cloudinary).
- Capa de Persistencia (Datos):
 - Almacenamiento Estructurado: Base de datos PostgreSQL alojada en Neon, gestionada mediante Prisma ORM para operaciones CRUD seguras y tipadas.
 - Almacenamiento de Blobs: Cloudinary se utiliza como repositorio de objetos para los archivos multimedia (imágenes y audio), almacenando en la base de datos únicamente las referencias (URLs públicas y IDs).

[Figura II.3. Diagrama de Arquitectura del Sistema] (Nota para el estudiante: Dibuja un diagrama de bloques. Al centro pon "Next.js App (Frontend + API)". Con flechas bidireccionales conéctalo a: "Neon (PostgreSQL)."a)bajo, Cloudinary."a la derecha, "Firebase Auth."a la izquierda y IA Service."arriba).

2.2.5. Diseño del Modelo de Datos

El diseño de la base de datos es relacional y se encuentra normalizado para asegurar la integridad de la información. El modelo se implementó utilizando el esquema de Prisma, definiendo las siguientes entidades principales:

- User: Entidad central que almacena credenciales, roles y referencias a proveedores de identidad.
- Asset: Representa el recurso digital. Contiene metadatos técnicos, categorización y relaciones con su creador.
- AssetImage / AssetTag: Entidades auxiliares para gestionar propiedades específicas de las imágenes y el sistema de etiquetado para búsquedas.

- Interacciones (Like, Coment, Report): Tablas relacionales que vinculan usuarios con activos, permitiendo la trazabilidad de la actividad social.
- Notifications: Entidad transaccional diseñada para gestionar la comunicación asíncrona con el usuario sobre eventos relevantes.

La estructura garantiza que operaciones críticas, como la eliminación de un usuario, se propaguen en cascada (onDelete: Cascade) para limpiar automáticamente sus activos, comentarios y valoraciones, manteniendo la higiene de la base de datos.

[Figura II.4. Diagrama Entidad-Relación (DER)] (Nota: Aquí puedes usar una captura del diagrama que genera Prisma Studio o dibujar el esquema de tablas: User 1—N Asset, Asset 1—N Like, Asset 1—N Coment, etc. Asegúrate de mostrar las claves foráneas).

2.3. Epigrafe 3

2.3.1. Diseño de mecanismos de datos, implementación e interfaces de usuario

La eficiencia operativa de Assets Studio reside en su capacidad para gestionar grandes volúmenes de datos multimedia sin comprometer el rendimiento del cliente final. Para ello, se diseñó una arquitectura de datos híbrida que separa el almacenamiento de metadatos relacionales del almacenamiento de objetos binarios (BLOBs), optimizando tanto la transmisión como el procesamiento.

2.3.2. Mecanismos de Almacenamiento y Persistencia

La estrategia de persistencia se implementó siguiendo un modelo distribuido para garantizar la escalabilidad y la integridad referencial:

- Almacenamiento Relacional (Metadatos): Se utilizó PostgreSQL desplegado en Neon (Serverless). Este mecanismo gestiona la estructura lógica del negocio: usuarios, relaciones sociales (likes, comentarios), y los punteros a los archivos. La interacción con la base de datos se abstrae mediante el ORM Prisma, lo que permite definir un esquema tipado y seguro.
- Almacenamiento de Objetos (Media): Los archivos pesados (imágenes y audio) se delegan a Cloudinary. Esta plataforma no solo almacena el archivo raw, sino que realiza transformaciones automáticas (formato, calidad, redimensionamiento) antes de entregarlo al cliente, cumpliendo con el requisito de optimización de ancho de banda.

El siguiente fragmento de código muestra la definición del esquema en Prisma (schema.prisma) utilizado para vincular ambas fuentes de datos, donde el modelo Asset almacena la referencia src proveniente de la nube y la relaciona con el usuario y las interacciones locales:

```
codePrisma
```

2.3.3. Mecanismos de Procesamiento y Transmisión

La transmisión de datos entre el cliente (navegador) y el servidor se realiza mediante API Routes de Next.js, utilizando el protocolo HTTPS y el formato JSON para el intercambio de mensajes.

El flujo de procesamiento para la creación de un nuevo asset (ya sea subido o generado por IA) sigue una lógica transaccional para evitar datos huérfanos:

- Validación de Sesión: Se verifica el token de autenticación del usuario (Firebase).
- Carga/Generación: El archivo se procesa y se sube a Cloudinary.
- Transacción de Base de Datos: Una vez obtenida la URL segura, se ejecuta una operación de escritura en PostgreSQL.

A continuación, se presenta un ejemplo de la lógica de implementación utilizada en el controlador del backend para registrar un asset, asegurando que se guarden tanto la imagen como sus etiquetas asociadas:

codeJavaScript

2.3.4. Diseño de Interfaces Gráficas de Usuario (GUI)

a interfaz de usuario se diseñó priorizando la usabilidad y la carga progresiva (Lazy Loading). Se implementó un diseño responsivo que se adapta a dispositivos móviles y de escritorio, utilizando una paleta de colores oscuros para resaltar el contenido visual (los assets) y reducir la fatiga visual en sesiones de trabajo prolongadas.

Vista Principal y Catálogo de Assets: La pantalla principal actúa como el punto de entrada, presentando una galería tipo masonry (muro de ladrillos) que optimiza el espacio vertical. Incluye una barra de búsqueda semántica y filtros por categoría.

[Figura II.5. Interfaz de la Página Principal y Galería de Assets] (Nota: Captura de pantalla donde se vea la barra de búsqueda arriba y la cuadrícula de imágenes abajo).

Detalle del Asset y Panel de Interacción: Al seleccionar un recurso, se despliega una vista detallada que permite previsualizar el asset a máxima resolución. En esta interfaz se integran los componentes sociales: botón de "Me gusta", sección de comentarios en tiempo real y opciones de descarga.

[Figura II.6. Interfaz de Detalle de Asset con Comentarios y Opciones] (Nota: Captura donde se vea una imagen grande, y al lado o abajo los comentarios de usuarios y el botón de descarga).

Panel de Gestión y Generación con IA: Esta interfaz permite al usuario autenticado subir sus propios archivos o utilizar la herramienta de generación. Se diseñó un formulario intuitivo que guía al usuario en el etiquetado del recurso. En el caso de la IA, incluye un campo de texto para el prompt y un área de previsualización para decidir si descartar o guardar el resultado.

[Figura II.7. Interfaz de Generación de Assets mediante IA] (Nota: Captura del formulario donde pones el texto para la IA y te muestra la imagen generada).

2.4. Epígrafe 4

2.4.1. Diseño de la gestión de errores y despliegue de la solución

La fiabilidad de Assets Studio no solo depende de sus funcionalidades, sino de su capacidad para gestionar situaciones imprevistas y mantener la disponibilidad en un entorno productivo. En este epígrafe se detalla la estrategia de tolerancia a fallos y el diseño de la arquitectura de despliegue en la nube.

2.4.2. Estrategia de Gestión y Tratamiento de Errores

El sistema implementa una estrategia de "defensa en profundidad" para el manejo de excepciones, abordando los errores en tres niveles: interfaz de usuario, lógica de servidor y base de datos.

Nivel de Interfaz (Frontend):

- Validación Preventiva: Se implementaron validaciones de formulario en tiempo real para evitar el envío de datos incorrectos (ej. formatos de archivo no soportados o campos vacíos).
- Feedback Visual: Ante fallos en la carga de assets o errores de red, el sistema despliega notificaciones tipo Toast (alertas emergentes) que informan al usuario del estado de la operación sin interrumpir su navegación.
- Páginas de Error (Fallbacks): Se diseñaron páginas estáticas personalizadas (404 Not Found y 500 Server Error) para mantener la identidad visual de la marca incluso cuando el servidor no responde.

Nivel de Servidor y API (Backend):

- Bloques Try/Catch: Todas las rutas de la API (Next.js API Routes) envuelven sus operaciones críticas en bloques de manejo de excepciones. Esto asegura que si falla un servicio externo (como Cloudinary o la IA), el servidor no se detenga, sino que devuelva un código HTTP adecuado (ej. 503 Service Unavailable o 400 Bad Request).
- Manejo de Errores de Base de Datos: Prisma ORM gestiona los errores de conexión con Neon. Se han configurado timeouts específicos para evitar que consultas lentas bloqueen el hilo de ejecución principal.

Reporte Participativo de Errores:

- Aprovechando el modelo de datos Report definido en el esquema, se habilitó un mecanismo para que los usuarios finales reporten anomalías.
- Los usuarios pueden señalar assets corruptos o fallos funcionales, creando un registro directo en la base de datos para su posterior revisión por un administrador.

[Figura II.8. Diagrama de Flujo de Gestión de Excepciones] (Nota para el estudiante: Dibuja un diagrama sencillo: Usuario realiza acción ->¿Error en datos? (Sí: Mensaje de alerta / No: Enviar al Servidor) ->Servidor procesa ->¿Fallo externo? (Sí: Retornar error 500 / No: Éxito). Incluye una cajita que diga Registro en Log").

2.4.3. Diseño del Despliegue (Deployment)

La arquitectura de despliegue de Assets Studio sigue un modelo Serverless y distribuido, diseñado para maximizar la disponibilidad y minimizar los costes de mantenimiento de infraestructura. La solución se despliega aprovechando la integración nativa del ecosistema Next.js.

El flujo de despliegue continuo (CI/CD) consta de las siguientes etapas:

- Entorno de Desarrollo Local: Los desarrolladores trabajan sobre ramas de características (feature branches), ejecutando la aplicación en un entorno Node.js local que conecta con una base de datos de desarrollo en Neon.
- Control de Versiones: El código se gestiona en un repositorio remoto (GitHub).
- Build y Despliegue Automático:
 - Al realizar un push a la rama principal (main), se dispara el proceso de construcción (build) en la plataforma de hosting (Vercel).
 - Durante el build, Next.js optimiza las imágenes, compila las rutas estáticas y verifica las dependencias.
 - Las migraciones de base de datos (prisma migrate deploy) se ejecutan para asegurar que el esquema en Neon coincida con la nueva versión del código.
- Producción: Una vez finalizado el proceso, la nueva versión se distribuye automáticamente a través de una red de entrega de contenido (CDN) global.

Componentes de la Infraestructura de Producción: la Infraestructura de Producción: la Infraestructura de Producción: la Infraestructura de Producción: la Infraestructura de Producción: la Infraestructura de Producción: la Infraestructura de Producción:

- Servidor Web/App: Vercel (Ejecución de Next.js y Serverless Functions).
- Base de Datos: Neon (PostgreSQL gestionado).
- Almacenamiento Media: Cloudinary (CDN para imágenes y audio).
- Autenticación: Firebase Identity Platform.

[Figura II.9. Diagrama de Despliegue de la Solución] (Nota: Dibuja un diagrama de despliegue UML. Muestra nodos (cubos 3D): Cliente (Navegador)", "Servidor de Aplicaciones (Vercel)", "Servidor de Base de Datos (Neon)", "Servidor de Archivos (Cloudinary)". Conecta los nodos con líneas que indiquen el protocolo, ej: "HTTPS/JSON").

2.4.4. Gestión de Variables de Entorno y Seguridad

Para garantizar la seguridad del despliegue, ninguna credencial se almacena en el código fuente. Se utiliza un sistema de variables de entorno (.env) inyectadas en tiempo de ejecución. Las variables críticas configuradas en el entorno de producción incluyen:

- DATABASE URL: Cadena de conexión encriptada a la instancia de Neon.
- NEXT PUBLIC CLOUDINARY CLOUD NAME: Identificador público para la carga de imágenes.
- CLOUDINARY API SECRET: Llave privada para firmar las subidas de archivos (backend).
- FIREBASE CLIENT EMAIL y PRIVATE KEY: Credenciales para validar tokens de sesión de Google/GitHub.

2.5. Conclusiones del Capítulo

Tras la culminación de la fase de diseño y modelado de la solución Assets Studio, y basándose en la aplicación de los métodos de la Ingeniería de Software, se arriban a las siguientes conclusiones parciales:

- El modelado de los procesos de negocio mediante el Lenguaje Unificado de Modelado (UML) permitió descomponer la complejidad de la gestión de activos digitales en flujos de trabajo optimizados. Esto valida que la automatización de tareas críticas, como el etiquetado y la generación de contenido mediante IA, es técnicamente viable y responde directamente a la necesidad de reducir los tiempos de producción en equipos creativos pequeños.
- La arquitectura de software diseñada, basada en un enfoque Serverless y distribuido (Next.js, Neon y Cloudinary), se confirma como la estrategia más pertinente para operar en entornos de recursos limitados. Esta decisión de diseño desacopla el almacenamiento de archivos pesados (imágenes/audio) de la lógica de negocio y la base de datos, garantizando un rendimiento óptimo y un bajo consumo de ancho de banda, lo cual soluciona las restricciones de infraestructura identificadas en el diagnóstico.
- La Ingeniería de Requisitos aplicada garantizó que la selección del stack tecnológico no fuera arbitraria, sino que respondiera a atributos de calidad específicos como la escalabilidad y la mantenibilidad. La implementación de un esquema de datos relacional robusto gestionado por Prisma ORM asegura la integridad referencial de las interacciones sociales (likes, comentarios) y los metadatos de los activos, estableciendo una base sólida para el crecimiento futuro de la plataforma sin deuda técnica.
- El diseño de interfaces y mecanismos de despliegue demuestra que es posible integrar tecnologías avanzadas de generación artificial y almacenamiento en la nube en una solución accesible y segura. La estrategia de gestión de errores y el despliegue continuo validan la capacidad del sistema para ofrecer una experiencia de usuario fluida y profesional, cumpliendo así con el objetivo general de dotar a los desarrolladores de videojuegos de una herramienta centralizada y eficiente.

Escribir aquí el capítulo 3. Otra prueba más [engine](#).

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Conclusiones

Escribir aquí las conclusiones.

Recomendaciones

Escribe aquí las recomendaciones de tu trabajo.

Apéndices



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
 Universidad de las Ciencias Informáticas, La Habana, Cuba
 Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
 Reorientaciones: sí
 Cambios de herramientas: no
 Vértices: 17
 Aristas: 136
 Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
 Reorientaciones: 2

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
 Reorientaciones: 3

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +y)-(17, +y)-(1, +y)

Hormiga: 3
 Reorientaciones: 3

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
 Reorientaciones: 2

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
 Reorientaciones: 3

Salida:
 (11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
 Reorientaciones: 2

Salida:
 (11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Otro reporte

B.1. Otra sección de muestra

De una sola sentencia

B.2. Otro ensamble Industrial



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Hormiga: 7

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)-(3, -z)

Hormiga: 8

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(12, -z)-(11, -z)

Hormiga: 9

Reorientaciones: 3

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(10, -z)-(11, -z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(17, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(12, -x)

Hormiga: 10

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)

Hormiga: 11

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)

Secuencia de salida premiada**Reorientaciones: 2****Salida:**

(8, -x)-(15, -x)-(14, -x)-(16, -x)-(10, -z)-(11, -z)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(6, +z)-(13, +z)-(3, +z)-(17, +z)-(2, +z)-(1, +z)